COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/23/26
NAME:	Garcia, Dennis James T. Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

ILO A

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

Algorithms

Insertion Sort:

```C/C++
template <typename T>
void insertionSort2(T arr[], int maxSize){
    for (int k = 1; k < maxSize; k++){
        T temp = arr[k];
        int j = k - 1;
        for (; j >= 0 && temp <= arr[j]; j--){
            arr[j + 1] = arr[j];
        }
        arr[j + 1] = temp;
    }
}
```

Shell Sort:

```C/C++
template<typename T>
void shellSort(T arr[], int size){
    for (int gap = size / 2; gap > 0; gap /= 2){
        for (int i = gap; i < size; i++){
            int temp = arr[i];
```

```
            int j = i;

            while (j >= gap && arr[j - gap] > temp){
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}
```

Output

	Best Case	Worst Case	Analysis
Insertion Sort	**O(n)**	**O(n^2)**	Insertion Sort works faster if the data is being sorted, but it works slower if the data is in reverse or random order. The amount of elements being sorted will gradually decrease the performance of the algorithm.
Shell Sort	**O(n log n)**	**O(n^2)**	Shell sort is usually faster than the Insertion because it compares the elements that are far apart using a gap based pattern rather than comparing the next element that is near each other. The large amount of data will affect the performance of the algorithm. Knowing that this doesn't compare elements next to each other should make this algorithm more resource efficient than Insertion Sort.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

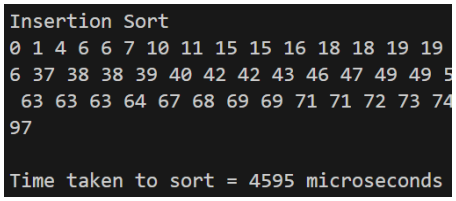
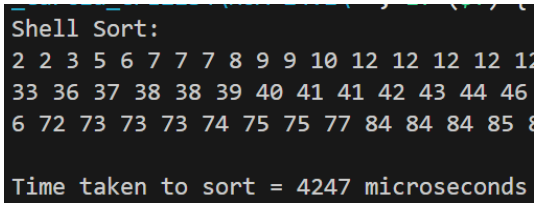
	Insertion Sort	Shell Sort
Best Case	n = 100	n = 100
		
	Execution Time: 4595 microseconds	**Execution Time: 4247 microseconds**

Table 14-2. Best-Case Analysis using Experimental Tools

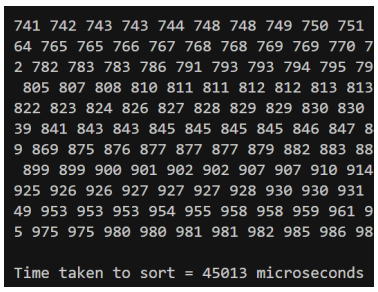
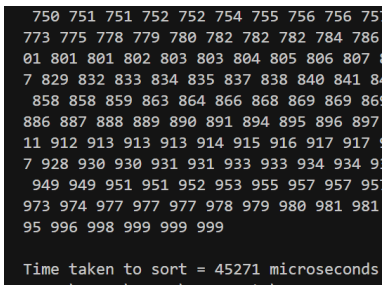
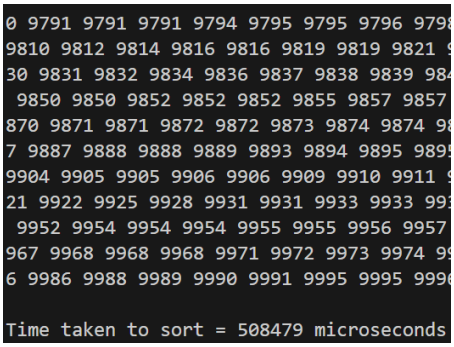
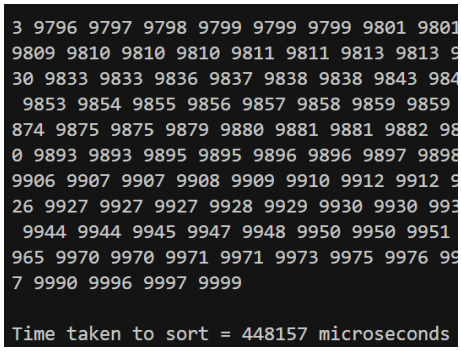
Worst Case	n = 1000	n= 1000
		
	Execution Time: 45013 microseconds	**Execution Time: 45271 microseconds**
	n = 10, 000	n = 10, 000
		
	Execution Time: 508479 microseconds	**Execution Time: 448157 microseconds**

Table 14-2. Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A.1

Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis

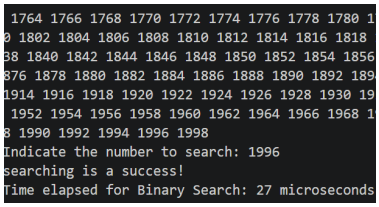
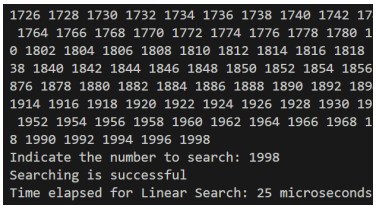
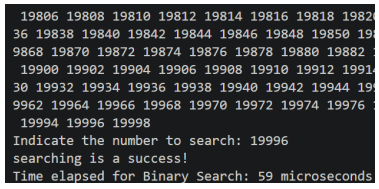
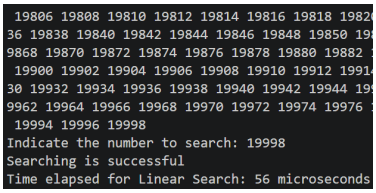
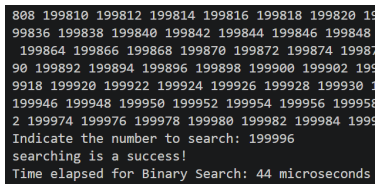
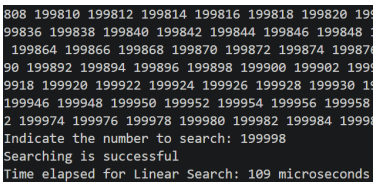
Linear Search:
Start from the first element, then compare the current element with the target. If they match, return success. If not, move to the next element. Repeat until the target is found or the list ends. If the target is not found, return unsuccessful.

Binary Search:
Set the lower and upper boundaries.Find the middle element. If the middle element matches the target, return success. If the target is greater, search the right half. If the target is smaller, search the left half. Repeat until the target is found or the search range is empty. If the target is not found, return unsuccessful.

- Time and Space Complexities

Searching Techniques	Time Complexity		Space Complexity
Linear	Best Case: **O(1)**	Worst Case: **O(n)**	**O(1)**
Binary	Best Case: **O(1)**	Worst Case: **O(n log n)**	**O(1)**

Empirical Analysis

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000		
	Execution Time: 27 microseconds	**Execution Time: 25 microseconds**
10,000		
	Execution Time: 59 microseconds	**Execution Time: 56 microseconds**
100,000		
	Execution Time: 44 microseconds	**Execution Time: 109 microseconds**

Analysis:

- Garcia:
  Judging by the screenshots and execution time, it seems that there's a limit that the linear search can reach at peak performance then its performance will gradually decrease once it reaches large amounts of data. Binary search seems slower at first but when it searches around a huge chunk of data which in the screenshots' example is the input size value of one-hundred thousand; you can see that binary search is faster by half of the linear search's result. Meaning that some algorithms are really good for large amounts of data.

- Hare:
  I noticed that larger amounts of data affects the performance of both algorithms, since linear search scans elements one by one, at larger values it would take too long, meanwhile it compares the target element to the middle element and divides the array by half which would make sense why at larger values the binary search is faster by half of the result of the linear search algorithm.

Conclusion	
Summary of the lessons learned	➢ Garcia: Algorithms performance will change depending on the size of data inputted. Some algorithms seem to have a limit on the amount of data they can sort or search within on peak performance, meaning that the larger sizes or amount of data can affect their performance slowing it down while other algorithms can handle higher amounts or sizes of data without significantly affecting their performance. ➢ Hare: I learned the importance of analyzing algorithms to understand how their performance changes as the input size increases.
Analysis of the procedure	➢ Garcia: Based on the empirical results or testing for the procedure, It seems that shell short can manage bigger amounts of data keeping it consistent regardless of the size of data, although the limit was not tested yet. Insertion sort can keep up between input sizes of a thousand but loses to the shell sort at higher values below a thousand. ➢ Hare: I observed that shell sort can sort data more efficiently by comparing elements with large gaps before sorting the nearby elements, it's processing the large input size better but Insertion sort runs the small size better based on the output.
Analysis of the supplementary activity	➢ Garcia: Similar to the analysis of the insertion sort and shell sort or sorting algorithms, the searching algorithms' performance will be affected by the input size. Binary search seems to handle larger amounts of data better than linear search. For the implementation, the chrono timer should only start when the function for searching an integer was called. Reusing the code from HoA 6.1, that function contains the result of the generated array and the chosen integer to find. Therefore the timer starts once the function is about to be called then ends afterwards. ➢ Hare: I noticed that both searching algorithms can find the target, but their execution time changes depending on the number of elements. Linear search is easier to understand because it checks each element one by one while binary search reduces the search range by half in every step. Based on our output, binary search becomes more useful for larger datasets, although the actual execution time may vary depending on the test and computer performance.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	➢ Garcia: I just need to remember the fundamentals again since the majority of the code that was mentioned was reused from preliminaries and midterms of this school year. Especially getting reintroduced to dynamically allocated arrays again. ➢ Hare: I think I understood the general concepts but i need to improve my on time complexity and check more examples about it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The compiler was throwing error issues due to large input sizes but we were reintroduced with the concept of dynamically allocated arrays via new int. The majority of the code was not modified and was already given (slightly modified main.cpp), We just reimplemented a pointer and new int for the dataset arrays.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.